MVP Hotel

LAZARO SOFTWARE

Módulo hotelero para Lazaro Software

Documento técnico y propuesta: reservas por fechas sin sobreventa, tarifas por temporada, folio de consumos y operación diaria de recepción.

Versión
MVP funcional · octubre de 2026

Stack
NestJS 11 · React 19 · PostgreSQL 16

Calidad
196 pruebas automáticas en verde

Contenido

1 Resumen ejecutivo
2 El problema
3 La propuesta y el alcance del MVP
4 Recorrido funcional
5 Arquitectura
6 Stack y módulos
7 Invariantes y dónde se protegen
8 Decisiones de diseño
9 Motor de tarifas
10 Ciclo de vida, folio y estadía
11 Modelo de datos
12 API REST
13 Seguridad y control de acceso
14 Calidad y pruebas
15 Ejecución y despliegue
16 Hoja de ruta
17 Decisiones a validar, riesgos y supuestos
Anexo A · Reglas de negocio R1–R36
Anexo B · Guion de la demo

1 Resumen ejecutivo

Lazaro Software atiende hoy a los hoteles con su modelo de restaurantes, donde **una habitación es una mesa**. Ese modelo no permite vender noches a futuro, cobrar distinto en temporada alta ni acumular consumos durante una estadía. Este MVP es un **módulo hotelero propio**, funcional de punta a punta (interfaz → API → base de datos), cuyo objetivo central es **reservar una habitación por fechas sin que pueda venderse dos veces**.

36 reglas de negocio documentadas (R1–R36)	**196** pruebas automáticas, todas en verde	**0** sobreventas posibles, incluso con peticiones simultáneas	**1** comando para levantar la demo completa con Docker

Qué entrega el MVP

- **Reservas sin sobreventa**, garantizadas por una restricción de exclusión de PostgreSQL y probadas con 10 peticiones concurrentes.
- **Tarifas reales**: temporadas alta, media y baja calculadas (Semana Santa y festivos de Colombia con Ley Emiliani), persona adicional, IVA del 19 % y desglose noche a noche congelado en la reserva.
- **Estadía completa**: check-in con registro hotelero, folio de consumos cargados a la habitación y check-out que liquida todo con su medio de pago; salida anticipada.
- **Operación de recepción**: panel "Hoy", calendario de ocupación, ciclo de limpieza, no-shows automáticos, huéspedes únicos, usuarios con roles y auditoría de cada acción.

Qué se propone

Validar con el empresario las políticas por defecto (sección 17) y avanzar por la hoja de ruta: refactor hexagonal, integración con el POS de Lazaro para cargar consumos al folio, reserva web con pagos (Wompi), canales de venta (Booking, Expedia, Airbnb vía Channex), WhatsApp y despliegue multi-hotel. La integración con el sistema actual de Lazaro (.NET) se haría por la API HTTP documentada.

2 El problema

Una mesa se ocupa y se libera en el momento. Una habitación, en cambio, **se compromete a futuro por un rango de fechas**, tiene un precio que depende de la fecha y acumula consumos durante días. Con el modelo actual, el hotel no puede:

Saber qué está libre a futuro

No hay disponibilidad por rango de fechas: solo "ocupada" o "libre" ahora.

Evitar vender dos veces

Una sobreventa obliga a reubicar al huésped a costo propio y daña la reputación en Booking.

Cobrar temporada alta

El principal reclamo actual: el precio no cambia con la temporada ni con las personas adicionales.

Cargar consumos a la habitación

Lo que el huésped consume en el restaurante no llega a una cuenta única de la estadía.

3 La propuesta y el alcance del MVP

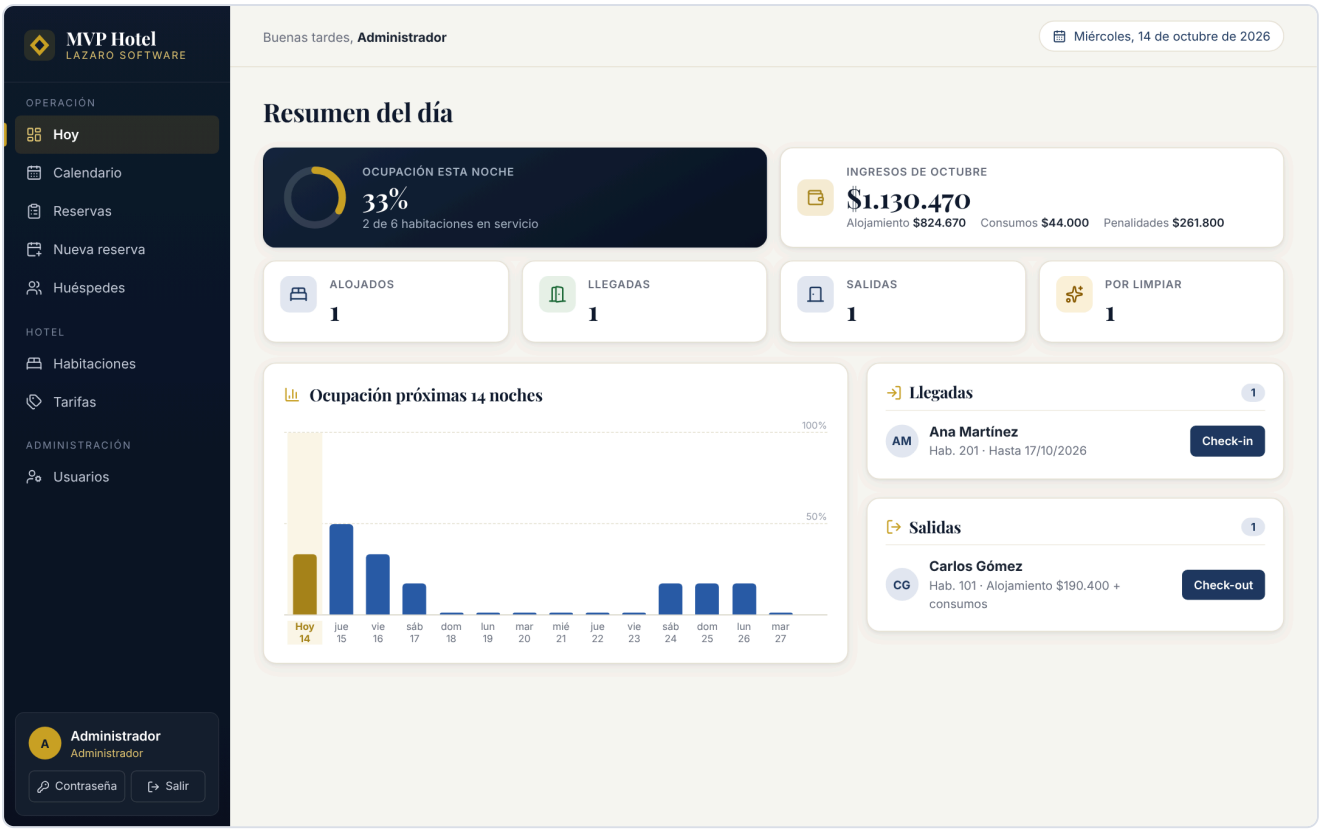
Un módulo independiente con su propio stack, diseñado para integrarse con Lazaro por HTTP. Su alcance actual:

Área	Incluido en el MVP
Reservas	Disponibilidad por fechas y capacidad, creación con precio congelado, cancelación gratuita o con penalidad, no-show automático, listado paginado con búsqueda, historial de auditoría.
Tarifas	Tipos de habitación (tarifa base, capacidad incluida y máxima, cargo por persona adicional), temporadas calculadas por año, IVA con exención para pasaporte, desglose por noche.
Estadía	Check-in con registro hotelero (nacionalidad, procedencia, motivo), folio de consumos con anulación, check-out con liquidación y medio de pago, salida anticipada.
Habitaciones	Estados libre → ocupada → en limpieza → libre, fuera de servicio con aviso de reservas afectadas.
Huéspedes	Huésped único por tipo y número de documento normalizado; búsqueda, ficha e historial; registro hotelero del día.
Operación	Panel "Hoy" (ocupación, llegadas, salidas, por limpiar, ingresos del mes) y calendario de ocupación.
Acceso	Inicio de sesión obligatorio, roles recepcionista y administrador, gestión de usuarios, cambio de contraseña.

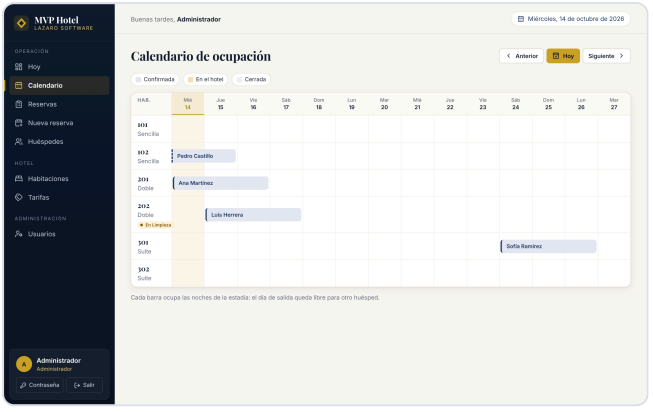
Fuera del alcance actual (hoja de ruta): pagos en línea y reserva web, canales OTA, WhatsApp, POS automático, envío directo a TRA / Migración, multi-hotel y despliegue en la nube.

4 Recorrido funcional

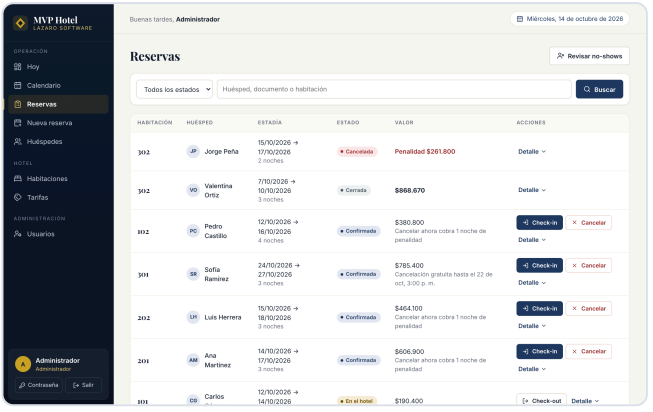
Capturas del MVP en ejecución con los datos de demostración que genera el *seed* (fechas relativas a hoy).



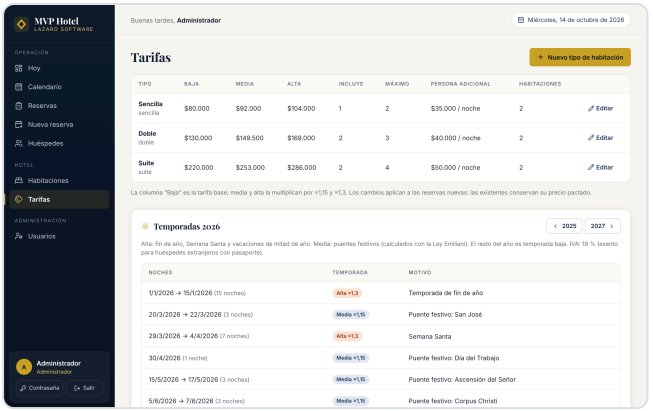
Panel "Hoy": ocupación de esta noche, ingresos del mes desglosados, llegadas, salidas, habitaciones por limpiar y ocupación de las próximas 14 noches.



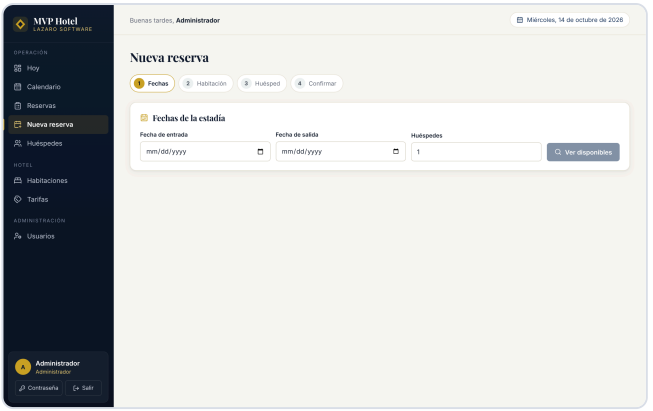
Calendario de ocupación: habitaciones × días; el día de salida queda libre para otro huésped.



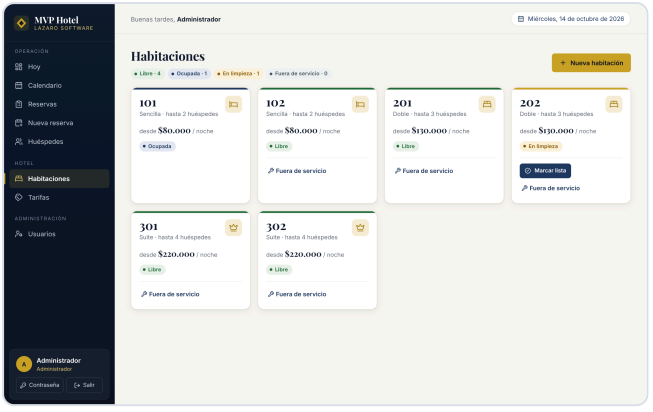
Reservas: filtro por estado, búsqueda, penalidad o cancelación gratuita calculada, check-in y check-out.



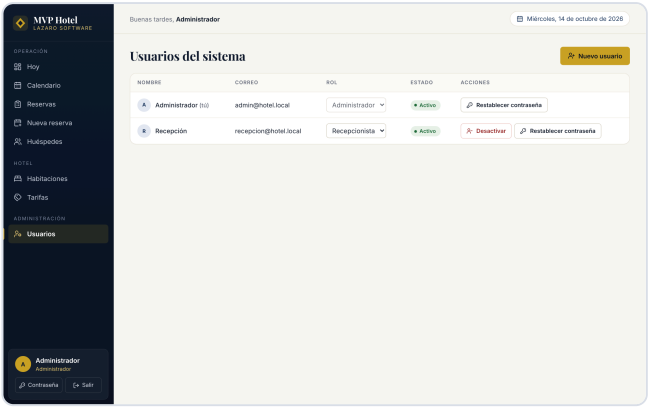
Tarifas: tipos de habitación editables y temporadas del año calculadas.



Nueva reserva: fechas, huéspedes y habitaciones disponibles con cotización.



Habitaciones: estado operativo, limpieza y fuera de servicio.



Usuarios: roles, desactivación y restablecimiento de contraseña.

5 Arquitectura



- **Frontend**: SPA en React. No tiene reglas de negocio propias: muestra lo que calcula el backend (precios, penalidades, ocupación) y oculta acciones según los permisos, pero **la protección real está en el backend**.
- **Backend**: API REST en NestJS con módulos por área del negocio. Toda petición pasa por una guardia global (seguro por defecto). Documentación interactiva en `/docs` (Swagger).
- **Base de datos**: PostgreSQL, accedida con Prisma. Es obligatoria: la garantía contra la sobreventa es una restricción de PostgreSQL.
- **Tarea programada**: cada 10 minutos marca los no-shows, cobra la penalidad y libera las noches restantes.

6 Stack y módulos

Backend

NestJS 11, TypeScript, Prisma, `@nestjs/schedule`, JWT, helmet, Swagger.

Frontend

React 19, Vite, TypeScript; identidad propia (Inter + Playfair Display).

Datos

PostgreSQL 16 con `btree_gist`; 10 migraciones versionadas.

Infraestructura

Docker Compose: solo la base para desarrollo, o toda la app con el perfil `demo`.

Pruebas

Jest + supertest contra PostgreSQL real; Vitest en el frontend.

Estilo

ESLint y Prettier compartidos entre backend y frontend.

Módulo	Responsabilidad
`auth`	Login, sesión JWT (8 h), guardia global, permisos por rol, límite de intentos de login
`usuarios`	Gestión de usuarios del hotel (crear, rol, desactivar, restablecer contraseña)
`habitaciones`	Habitaciones, disponibilidad con precio, estados (limpieza, fuera de servicio)
`tarifas`	**Dominio puro**: calendario de Colombia, temporadas y motor de tarificación; tipos de habitación

Módulo	Responsabilidad
`reservas`	Ciclo de vida: crear, check-in, check-out, cancelar, no-show; auditoría; registro hotelero
`folio`	Consumos cargados a la habitación y liquidación del check-out
`huespedes`	Huésped único por documento, búsqueda e historial
`panel`	Vistas de operación: panel "Hoy" y calendario de ocupación
`hotel`	Políticas del hotel (horarios, plazos, multiplicadores, IVA) y el reloj inyectable

Los endpoints piden **permisos**, no roles (`auth/permisos.ts`): cambiar qué puede hacer un rol es editar una tabla, sin tocar los controladores.

7 Invariantes y dónde se protegen

La pregunta que guía el diseño: *¿qué regla del negocio nunca se puede romper, y qué la garantiza aunque dos personas actúen al mismo tiempo?*

Invariante	Dónde se protege	Cómo se prueba
No sobreventa: una habitación no tiene dos reservas vigentes que se solapen	Restricción `EXCLUDE USING gist` en PostgreSQL (atómica); el servicio hace una verificación previa solo para responder un 409 claro	10 reservas simultáneas → 1 gana. **Sin la restricción, las 10 se crean**
La tarifa depende de la temporada de cada noche, las personas adicionales y el IVA	Motor de tarificación puro (`tarifas/`); el desglose se **congela** en la reserva	24 pruebas unitarias sin base de datos (Pascua, 18 festivos de 2026, temporadas, IVA, salida anticipada)
Solo se cargan consumos con la estadía activa	Transacción que **bloquea la fila de la reserva** (`SELECT … FOR UPDATE`)	Cargos y check-out simultáneos: el total siempre cuadra. **Sin el bloqueo falla 4 de cada 5 veces**
El check-out cierra y liquida el folio	Transacción única: alojamiento pactado + consumos vigentes, medio de pago, habitación a limpieza, evento de auditoría	Liquidación, anulaciones y rechazo de cargos posteriores
Solo se entrega una habitación lista	Actualización condicionada en el check-in ("solo si está libre"): también evita dos check-in a la vez	Ciclo libre → ocupada → en limpieza → libre
Cada acción queda atribuida a una persona	Evento de auditoría **en la misma transacción** que el cambio; un usuario con acciones no se puede borrar (FK `RESTRICT`)	Una acción rechazada no deja rastro; una exitosa siempre lo deja
Un huésped no se duplica	Índice único (tipo + número de documento normalizado) y manejo de la carrera de dos reservas simultáneas	Dos reservas a la vez del mismo huésped nuevo → un solo registro

La garantía contra la sobreventa

```
-- backend/prisma/migrations/20260928231947_init/migration.sql
CREATE EXTENSION IF NOT EXISTS btree_gist;

ALTER TABLE "reserva"
  ADD CONSTRAINT reserva_sin_sobreventa
  EXCLUDE USING gist (
    habitacion_id WITH =,
    daterange(fecha_entrada, fecha_salida, '[)') WITH &&
  )
  WHERE (estado IN ('confirmada', 'activa'));
```

El rango semiabierto `'[)'` deja libre el día de salida para el siguiente huésped (entrada 10, salida 15 = 5 noches). Solo bloquean las reservas confirmadas o activas: canceladas, cerradas y no-

shows liberan las fechas al instante. El servicio traduce el error de PostgreSQL a `409 Conflict` en lugar de un `500`.

Por qué no basta un `if`: Node.js es de un solo hilo, pero dos peticiones se intercalan en cada `await`. Se comprobó experimentalmente: sin la restricción, 10 reservas simultáneas de la misma habitación se creaban todas; con ella, solo una gana.

8 Decisiones de diseño

- **PostgreSQL y no otro motor**: la restricción de exclusión sobre rangos de fechas hace imposible la sobreventa, incluso con muchas peticiones concurrentes.
- **Precio congelado**: al reservar se guarda el desglose completo noche por noche (JSON). Si el hotel cambia sus tarifas, las reservas existentes conservan lo pactado y la penalidad usa el valor real de la primera noche.
- **Calendario calculado, no cargado**: la Pascua (algoritmo gregoriano) y los festivos con la Ley Emiliani se calculan para cualquier año; el hotel no mantiene un calendario.
- **Reloj inyectable**: las reglas que dependen de la hora (cancelación gratuita 48 h, no-show, "hoy" en Colombia UTC-5) se prueban fijando la hora.
- **Capas de NestJS con un núcleo de dominio puro**: `calendario-colombia.ts`, `temporadas.ts` y `motor-tarificacion.ts` no importan NestJS ni Prisma y se prueban sin base de datos. Es el primer paso del refactor hexagonal previsto antes de las integraciones externas.
- **Anular en vez de borrar**: consumos y usuarios se anulan o desactivan para no perder trazabilidad.
- **Políticas centralizadas**: horarios, plazos, multiplicadores e IVA viven en `hotel/politicas.ts`; son valores por defecto configurables por hotel.

9 Motor de tarifas

Baja · ×1,00
El resto del año. Es la tarifa base del tipo de habitación.

Media · ×1,15
Puentes festivos, calculados con la Ley Emiliani.

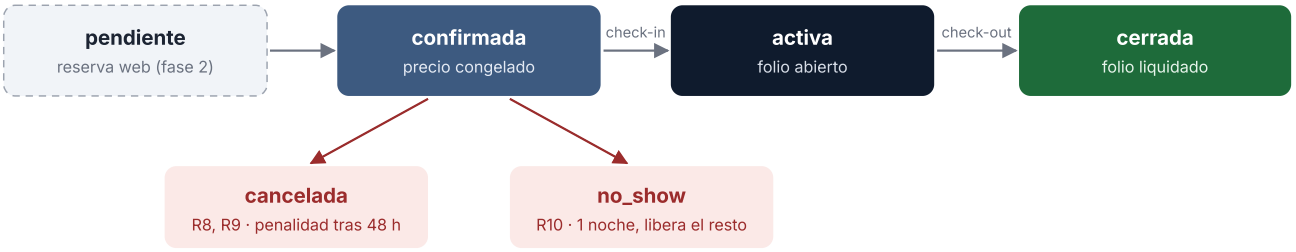
Alta · ×1,30
15 dic–15 ene, Semana Santa y 15 jun–15 jul.

Cálculo por noche: precio de la noche = tarifa base del tipo × multiplicador de la temporada de esa noche + cargo por persona adicional (sobre la capacidad incluida). Sobre el alojamiento se aplica el IVA del 19 %, exento cuando el huésped se registra con pasaporte (aproximación a "extranjero no residente", por validar con el contador).

Valores de demostración (seed), editables por el administrador:

Tipo	Baja	Media	Alta	Incluye	Máximo	Persona adicional
Sencilla	$80.000	$92.000	$104.000	1	2	$35.000 / noche
Doble	$130.000	$149.500	$169.000	2	3	$40.000 / noche
Suite	$220.000	$253.000	$286.000	2	4	$50.000 / noche

10 Ciclo de vida, folio y estadía



Reservar

`confirmada`. Entrada desde hoy, 1 a 30 noches, máximo 365 días de antelación, capacidad del tipo respetada. El precio y su desglose se congelan.

Check-in

`activa`, habitación `ocupada`. Desde la fecha de entrada; exige la habitación lista y los datos del registro hotelero. Abre el folio.

Folio

Consumos (minibar, lavandería, restaurante, late check-out, otro) solo con la estadía activa. Se anulan con motivo; nunca se borran.

Check-out

`cerrada`. Total = alojamiento pactado + consumos vigentes, con medio de pago (efectivo, tarjeta, transferencia). Habitación a `en_limpieza`.

Cancelación

Solo desde `confirmada`. Gratis hasta 48 h antes de las 15:00 del día de entrada; después, 1 noche de penalidad.

No-show y salida anticipada

Sin check-in al mediodía del día siguiente: `no_show`, 1 noche, se liberan las demás. Salida anticipada: se cobran solo las noches usadas.

Mientras no haya pagos en línea, las penalidades se registran como monto a cobrar. Todas las horas se calculan en hora de Colombia (UTC-5).

11 Modelo de datos

Entidad	Campos principales	Relaciones
TipoHabitacion	código, nombre, tarifa base, capacidad incluida, capacidad máxima, cargo por persona adicional	1 : N con Habitacion
Habitacion	número, estado (`libre` , `ocupada` , `en_limpieza` , `fuera_servicio`)	N : 1 TipoHabitacion · 1 : N Reserva
Reserva	fecha de entrada y salida, estado (`confirmada` , `activa` , `cerrada` , `cancelada` , `no_show`), número de huéspedes, valor de alojamiento, desglose (JSON), total, medio de pago	N : 1 Habitacion · N : 1 Huesped · 1 : N Cargo · 1 : N EventoReserva
Huesped	tipo de documento (CC, CE, pasaporte, TI, PPT), documento normalizado (único con el tipo), nombre, teléfono, nacionalidad, procedencia	1 : N Reserva
Cargo	categoría, concepto, total, anulado, motivo de anulación	N : 1 Reserva · N : 1 Usuario
EventoReserva	tipo (`creada` , `check_in` , `check_out` , `cancelada` , `no_show` , `cargo_agregado` , `cargo_anulado`), fecha, detalle	N : 1 Reserva · N : 1 Usuario (nulo = "Sistema")
Usuario	nombre, email, rol (`admin` , `recepcionista`), activo, hash de contraseña	1 : N EventoReserva y Cargo (FK RESTRICT)

12 API REST

Documentación interactiva en `http://localhost:3000/docs`. Todas las rutas exigen sesión salvo las marcadas como públicas; la columna indica el permiso requerido.

Método	Ruta	Permiso	Descripción
POST	`/auth/login`	`Pública`	Inicia sesión y devuelve el token JWT (8 h). Límite de intentos por IP.
GET	`/auth/yo`	`Sesión`	Usuario actual y sus permisos.
POST	`/auth/cambiar-contrasena`	`Sesión`	Cambio de la contraseña propia.
GET	`/salud`	`Pública`	Chequeo de salud, incluida la conexión a la base.
GET	`/habitaciones`	`habitaciones:ver`	Listado de habitaciones con su estado.
GET	`/habitaciones/disponibles`	`habitaciones:ver`	Disponibilidad por rango y número de huéspedes, con cotización.

Método	Ruta	Permiso	Descripción
POST	`/habitaciones`	`habitaciones:gestionar`	Crear habitación (admin).
PATCH	`/habitaciones/:id/estado`	`habitaciones:estado`	Marcar lista o fuera de servicio.
GET	`/tipos-habitacion`	`habitaciones:ver`	Tipos con tarifa base, capacidades y cargo adicional.
POST / PATCH	`/tipos-habitacion[/:id]`	`habitaciones:gestionar`	Crear y editar tipos y tarifas (admin).
GET	`/tarifas/temporadas?anio=`	`habitaciones:ver`	Calendario de temporadas calculado para el año.
GET	`/reservas`	`reservas:ver`	Listado paginado con filtro por estado y búsqueda.
POST	`/reservas`	`reservas:gestionar`	Crear reserva; precio y desglose congelados. 409 si se solapa.
POST	`/reservas/:id/check-in`	`estadias:gestionar`	Check-in con datos del registro hotelero.
POST	`/reservas/:id/check-out`	`estadias:gestionar`	Liquidación del folio con medio de pago.
POST	`/reservas/:id/cancelar`	`reservas:gestionar`	Cancelación gratuita o con penalidad.
POST	`/reservas/no-shows/procesar`	`no-shows:procesar`	Fuerza la revisión de no-shows (también corre cada 10 min).
GET	`/reservas/registro-hotelero`	`reservas:ver`	Registro hotelero del día, extranjeros marcados.
GET	`/reservas/:id/historial`	`reservas:ver`	Auditoría de la reserva.
GET	`/reservas/:id/folio`	`reservas:ver`	Alojamiento + consumos vigentes = total.
POST	`/reservas/:id/cargos`	`folio:gestionar`	Cargar consumo (solo con estadía activa).
POST	`/reservas/:id/cargos/:cargoId/anular`	`folio:gestionar`	Anular consumo con motivo.
GET	`/huespedes`	`reservas:ver`	Búsqueda de huéspedes.
GET	`/huespedes/documento/:tipo/:documento`	`reservas:ver`	Reconocer cliente por documento normalizado.
GET	`/huespedes/:id`	`reservas:ver`	Ficha e historial del huésped.

Método	Ruta	Permiso	Descripción
GET	`/panel/hoy`	`reservas:ver`	Ocupación, llegadas, salidas, por limpiar, ingresos del mes.
GET	`/panel/calendario`	`reservas:ver`	Calendario de ocupación habitaciones × días.
GET / POST / PATCH	`/usuarios[/:id]`	`usuarios:gestionar`	Gestión de usuarios (admin).
POST	`/usuarios/:id/restablecer-contrasena`	`usuarios:gestionar`	Restablecer contraseña (admin).

13 Seguridad y control de acceso

Permiso	Recepcionista	Administrador
Ver habitaciones y disponibilidad	✔	✔
Ver, crear y cancelar reservas	✔	✔
Check-in / check-out	✔	✔
Cargar y anular consumos del folio	✔	✔
Marcar habitaciones listas o fuera de servicio	✔	✔
Revisar no-shows	✔	✔
Crear habitaciones; editar tarifas y tipos de habitación	—	✔
Gestionar usuarios	—	✔

- **Seguro por defecto**: una guardia global protege todo salvo `POST /auth/login` y `GET /salud`.
- **Sesión**: JWT de 8 horas. Desactivar a un usuario o cambiarle el rol aplica **de inmediato**, aunque tenga la sesión abierta.
- **Contraseñas** con hash `scrypt`, mínimo 8 caracteres.
- **Límite de intentos de login**: 5 por minuto e IP (configurable); al superarlo responde `429`.
- **CORS** restringido a los orígenes del frontend y **cabeceras de seguridad** con `helmet`.
- Un administrador no puede desactivarse ni quitarse el rol a sí mismo: el hotel nunca se queda sin administradores.
- **Auditoría**: quién creó, canceló o hizo check-in/out de cada reserva y quién cargó o anuló cada consumo; los no-shows automáticos figuran como "Sistema".

14 Calidad y pruebas

160
pruebas e2e del backend contra PostgreSQL real (16 suites)

24
pruebas unitarias del dominio de tarifas, sin base de datos

12
pruebas del frontend (Vitest)

0
vulnerabilidades altas o críticas (`npm audit`)

Las 196 pruebas se ejecutaron en verde el 14 de octubre de 2026. Las e2e usan Jest + supertest contra una base aparte (`mvp_hotel_test`) porque la restricción `EXCLUDE` no se puede simular con mocks. Cubren: no-sobreventa (solapamientos, reservas contiguas, cancelada/cerrada no bloquea, 10 peticiones concurrentes, la base rechaza el solapamiento aunque se salte el servicio), check-in/check-out y sus transiciones inválidas, cálculo del total (cruce de mes, de año y año bisiesto), disponibilidad, cancelación y no-show, folio y concurrencia, estados de habitación, huéspedes, registro hotelero, salida anticipada, panel, seguridad y permisos (401 sin sesión, 403 por rol, desactivación inmediata).

Quedan avisos **moderados** de `js-yaml` / `sprintf-js` que llegan por `@nestjs/swagger` y por la cobertura de Jest; no aplican a esta API (no procesa YAML externo) y su corrección exige migrar a Nest 12.

15 Ejecución y despliegue

Demo con un solo comando (Docker)

```
cp backend/.env.example backend/.env        # solo la primera vez (cambiar JWT_SECRET)
docker compose --profile demo up -d --build
docker compose --profile demo run --rm backend node dist/prisma/seed.js
```

App en `http://localhost:8080` · API en `http://localhost:3000` · Swagger en `http://localhost:3000/docs`. El seed reinicia los escenarios de demostración con fechas relativas a hoy.

Desarrollo

```
docker-compose up -d                          # PostgreSQL en localhost:5432
cd backend && npm install && cp .env.example .env
npx prisma migrate dev && npm run seed && npm run start:dev
cd ../frontend && npm install && cp .env.example .env && npm run dev
```

Variables de entorno principales

Variable	Uso
`DATABASE_URL`	Conexión a PostgreSQL
`JWT_SECRET`	Firma de los tokens de sesión (valor largo y aleatorio en producción)
`ADMIN_*`, `RECEPCION_*`	Usuarios iniciales que crea el seed (solo desarrollo y demo)
`CORS_ORIGENES`	Frontends autorizados a llamar a la API
`LOGIN_INTENTOS_POR_MINUTO`	Límite de intentos de login por IP (5 por defecto)

16 Hoja de ruta

Las integraciones externas **no entran al dominio**: el dominio define una interfaz (puerto) y cada proveedor es un adaptador intercambiable. Orden propuesto, de lo barato y aislado a lo que toca infraestructura externa:

#	Fase	Qué aporta	Puerto
1	**Refactor hexagonal**	Extraer el resto del dominio como ya está el motor de tarifas.	—
2	**POS de Lazaro → folio**	El "cargar a la habitación" del restaurante llega automáticamente (hoy es manual).	`IPuntoDeVenta`
3	**Reserva web + pagos (Wompi)**	Anticipo del 30 % (PSE, Nequi, tarjeta); reserva `pendiente` que bloquea 30 min; el pago se confirma por webhook idempotente con firma verificada, nunca por el navegador.	`IPasarelaDePagos`
4	**Canales OTA y WhatsApp**	Booking, Expedia y Airbnb vía Channex como capa anticorrupción; confirmaciones por WhatsApp Business.	`IGestorDeCanales`, `INotificador`
5	**Producción**	Despliegue (DigitalOcean), observabilidad, modelo multi-hotel, envío a TRA y Migración.	—

17 Decisiones a validar, riesgos y supuestos

Decisiones que debe validar el empresario

1. ¿La política de cancelación de 48 h con penalidad de 1 noche le sirve, o cada hotel debe definir la suya?
2. ¿Los multiplicadores de temporada (×1,15 / ×1,30) son razonables, o sus hoteles manejan tarifas por temporada como precios fijos?
3. Salida anticipada: ¿se cobran las noches usadas o todas las reservadas?
4. ¿Qué porcentaje de anticipo exigen hoy sus clientes, si exigen alguno?
5. ⚖ IVA y exención para extranjeros: confirmar con el contador cómo lo factura hoy Lazaro.

Riesgos y supuestos

Riesgo o supuesto	Tratamiento
Exención de IVA aproximada por pasaporte	Parametrizado; validar con contador o asesor legal antes de producción.
Registro hotelero sin envío automático a TRA / Migración	El módulo deja los datos completos y el listado del día listo para reportar; integración en la hoja de ruta.
Políticas fijas para un solo hotel	Centralizadas en `hotel/politicas.ts`; el modelo multi-hotel las vuelve configurables por hotel.
Stack distinto al de Lazaro (.NET)	Integración por API HTTP documentada; las integraciones se aíslan detrás de puertos.
Sin pagos en línea	Las penalidades se registran como monto a cobrar; pagos con Wompi en la fase 3.
Modificación directa de reservas	Hoy: cancelar y crear una nueva (sin penalidad si la hace el hotel).

Conclusión. El MVP resuelve un problema real del negocio con cada regla crítica garantizada donde debe estarlo (la base de datos o una transacción) y demostrada con pruebas automáticas, incluso bajo concurrencia. La base técnica está lista para crecer hacia pagos, canales de venta y la integración con el POS de Lazaro.

Anexo A · Reglas de negocio R1–R36

Los valores numéricos (horas, porcentajes, montos) son **valores por defecto configurables por cada hotel**. Lo marcado con ⚖ debe validarse con un contador o asesor legal antes de usarse en producción. Fuente: `docs/REGLAS_DE_NEGOCIO.md`.

Reservas

ID	Regla	Por qué le importa al hotel	Estado en el MVP
R1	**Nunca se vende una habitación dos veces** en fechas que se solapen. No hay sobreventa intencional.	Una sobreventa obliga a reubicar al huésped en otro hotel a costo propio y daña la reputación (reseñas en Booking).	✔ **Sí** Garantizado por la base de datos y probado con concurrencia.
R2	La noche se cuenta de la fecha de entrada a la de salida: el día de salida queda libre para otro huésped (entrada 10, salida 15 = 5 noches).	Permite que una habitación se venda el mismo día que se desocupa.	✔ **Sí**
R3	No se puede reservar con fecha de entrada en el pasado.	Evita errores de digitación que bloquean habitaciones.	✔ **Sí**
R4	Estadía mínima de 1 noche y máxima de **30 noches**; antelación máxima de **365 días**.	Las estadías largas suelen negociarse aparte (tarifa mensual); reservas a años vista suelen ser errores.	✔ **Sí**
R5	**El precio se calcula y se congela al confirmar la reserva.** Si el hotel cambia sus tarifas después, las reservas existentes conservan el precio pactado.	Es lo que se le prometió al cliente.	✔ **Sí** Se guarda como `valorAlojamiento`.
R6	La reserva registra **número de huéspedes**, que no puede superar la capacidad máxima del tipo de habitación.	Base para cobrar personas adicionales (R12) y para no meter 4 personas en una sencilla.	✔ **Sí** Se registra; la disponibilidad excluye habitaciones sin capacidad.
R7	Cambiar fechas o habitación = cancelar y crear una nueva reserva (sin penalidad si la hace el hotel).	Mantiene simple el MVP; la modificación directa queda para después.	—

Cancelación y no-show

ID	Regla	Por qué le importa al hotel	Estado en el MVP
R8	Solo se cancela una reserva **confirmada** (antes del check-in). Una estadía activa no se cancela: se cierra con check-out.	Una vez el huésped está en la habitación ya hay consumo real.	✔ **Sí**
R9	Cancelación **gratuita hasta 48 h antes** de la fecha de entrada. Después, penalidad de **1 noche**.	Es la política más común en hoteles pequeños y en Booking; protege al hotel de cancelaciones tardías que ya no puede revender.	✔ **Sí**

ID	Regla	Por qué le importa al hotel	Estado en el MVP
R10	**No-show:** si el huésped no hace check-in antes de la hora de check-out del día siguiente a su entrada, la reserva pasa a `no_show`, se cobra **1 noche** de penalidad y **se liberan las noches restantes**.	Sin esta regla, una reserva fantasma bloquearía la habitación todas sus noches (y la regla R1 impediría venderla).	✔ **Sí** Tarea programada cada 10 min.
R11	Al cancelar o marcar no-show, las fechas quedan **disponibles de inmediato**.	Recuperar ingresos revendiendo la habitación.	✔ **Sí**

Tarifas

ID	Regla	Por qué le importa al hotel	Estado en el MVP
R12	La tarifa base se define **por tipo de habitación** (sencilla, doble, suite), con una **capacidad incluida** y una **capacidad máxima**. Cada persona adicional sobre la incluida paga un **cargo fijo por noche**.	Así publican los precios los hoteles; cambiar el precio de "todas las dobles" es una sola edición.	✔ **Sí** Tipos editables por el admin en la pestaña Tarifas.
R13	El año se divide en temporadas **alta / media / baja**, cada una con un **multiplicador** sobre la tarifa base (por defecto: baja ×1,00 · media ×1,15 · alta ×1,30).	Es el principal reclamo actual: no se puede cobrar más en temporada alta.	✔ **Sí**
R14	**Cada noche se cobra según su propia temporada.** Una estadía del 13 al 17 de diciembre cobra noches en media y en alta.	Es la forma justa y la que el huésped puede verificar en el desglose.	✔ **Sí** Desglose noche por noche, congelado en la reserva.
R15	Calendario inicial **fijo** definido por el hotel por año. Sugerido para Colombia: alta = 15 dic–15 ene, Semana Santa y 15 jun–15 jul; media = puentes festivos; baja = el resto. Semana Santa cambia cada año, por eso el calendario es **por año**.	Cubre los picos de turismo nacional; más adelante cada hotel lo configura.	✔ **Sí** Calculado: Pascua y festivos de Colombia (Ley Emiliani) para cualquier año.
R16	⚖ **IVA del 19 %** sobre el alojamiento, configurable por hotel. Los turistas **extranjeros no residentes** pueden estar exentos: se registra la nacionalidad del huésped para aplicarlo.	Cumplimiento tributario; la factura electrónica la sigue emitiendo Lazaro con su proveedor DIAN.	✔ **Sí** 19 %. ⚖ Exento en el MVP: quien se registra con pasaporte (aproximación a "extranjero no residente").
R17	El total siempre se muestra **desglosado**: noches × tarifa por temporada, personas adicionales, impuestos y cargos.	El recepcionista debe poder explicarle el cobro al huésped.	✔ **Sí** En la nueva reserva y en el detalle de cada reserva (cargos del folio: bloque C).

Estadía: check-in, folio y check-out

ID	Regla	Por qué le importa al hotel	Estado en el MVP
R18	Horarios por defecto: **check-in desde las 15:00, check-out hasta las 12:00** (configurables). Se puede hacer check-in desde la fecha de entrada.	Da tiempo de limpieza entre huéspedes.	◖ **Parcial** Valida la fecha de llegada; los horarios aún no se configuran.
R19	⚖ En el check-in se completan los datos del huésped (tipo y número de documento, nacionalidad, procedencia).	Colombia exige registrar a los huéspedes (Tarjeta de Registro Hotelero y, para extranjeros, reporte a Migración). La integración con esos sistemas queda para después; el módulo deja los datos listos.	✔ **Sí** El check-in exige nacionalidad, procedencia y motivo; "Registro hotelero" del día con los extranjeros marcados. Envío a TRA / Migración: hoja de ruta.
R20	Al hacer check-in se abre el **folio**: la cuenta de la estadía, que arranca con las noches.	Una sola cuenta por estadía.	✔ **Sí** Folio en el detalle de la reserva: alojamiento + consumos.
R21	**Solo se cargan consumos al folio mientras la estadía está activa** (después del check-in y antes del check-out).	Evita cargos a reservas que aún no llegan o que ya pagaron.	✔ **Sí** Seguro ante concurrencia: un cargo y el check-out simultáneos se ordenan (bloqueo de la reserva).
R22	Los cargos pueden venir de recepción (manuales: lavandería, late check-out, minibar) o del restaurante/bar de Lazaro ("cargar a la habitación"). En el MVP, solo manuales.	Es la integración clave con lo que Lazaro ya tiene.	◖ **Parcial** Manuales (minibar, lavandería, restaurante, late check-out); se anulan, no se borran. ⚖ Valor final con impuestos. POS: hoja de ruta.
R23	**El check-out liquida y cierra el folio**: total = noches + personas adicionales + impuestos + cargos − anticipo. Después del cierre no se admiten más cargos. Se paga completo en ese momento (sin pagos parciales) y se registra el medio de pago (efectivo, tarjeta, transferencia).	Cierre de caja claro; la factura sale de ese total.	✔ **Sí** Liquidación = alojamiento + consumos, con medio de pago (sin anticipo: aún no hay pagos en línea).
R24	Salida anticipada: se cobran las **noches realmente usadas** y se liberan las restantes.	Política amable y simple; se puede cambiar a "se cobra todo" si el hotel lo prefiere.	✔ **Sí** Del desglose pactado (temporada, adicionales, IVA); mínimo 1 noche; la salida se adelanta y lo pactado queda en la auditoría.

Habitaciones

ID	Regla	Por qué le importa al hotel	Estado en el MVP
R25	Estados: **libre** → **ocupada** (check-in) → **en limpieza** (check-out) → **libre** (housekeeping la marca lista).	No se debe entregar una habitación sucia al siguiente huésped.	✔ **Sí** El check-in exige la habitación lista; recepción la marca lista desde Habitaciones.
R26	Una habitación **fuera de servicio** (mantenimiento) no aparece como disponible.	Evita vender una habitación dañada.	✔ **Sí** Tampoco admite reservas nuevas; al sacarla de servicio se avisa cuántas reservas futuras hay que reubicar.

Pagos y anticipos (fase posterior, con Wompi)

ID	Regla	Por qué le importa al hotel
R27	Las reservas **hechas por recepción** se confirman de inmediato, sin anticipo (el hotel conoce al cliente o cobra en sitio).	Así trabajan hoy.
R28	Las reservas **web** exigen un **anticipo del 30 %** (o la primera noche). Mientras no se pague, la reserva queda `pendiente` y bloquea la habitación por **30 minutos**; si no se paga, expira y se libera.	Evita reservas falsas que bloquean inventario.
R29	El pago se considera hecho **solo cuando llega la confirmación de la pasarela** (webhook), nunca por lo que diga el navegador del huésped.	Seguridad: el navegador se puede manipular.

Usuarios y acceso

ID	Regla	Por qué le importa al hotel	Estado en el MVP
R30	Nadie usa el sistema sin iniciar sesión con su propio usuario.	Cada acción queda atribuible a una persona; un computador de recepción no queda abierto a cualquiera.	✔ **Sí**
R31	Dos roles: **recepcionista** (opera reservas, check-in/out, cancelaciones, no-shows) y **administrador** (además crea habitaciones y gestiona usuarios).	Recepción no debe poder cambiar el inventario ni dar accesos.	✔ **Sí**
R32	Al desactivar a un empleado (p. ej. cuando deja el hotel) pierde el acceso **de inmediato**, aunque tenga la sesión abierta. No se borra, para conservar el historial.	Seguridad ante rotación de personal.	✔ **Sí**
R33	Un administrador no puede desactivarse ni quitarse el rol a sí mismo.	El hotel nunca se queda sin alguien que gestione el sistema.	✔ **Sí**

ID	Regla	Por qué le importa al hotel	Estado en el MVP
R34	Registrar qué usuario creó, canceló o hizo check-in/out de cada reserva.	Auditoría y resolución de disputas con huéspedes.	✔ **Sí** Historial por reserva; los no-shows automáticos figuran como "Sistema".

Huéspedes

ID	Regla	Por qué le importa al hotel	Estado en el MVP
R35	Un huésped se identifica por **tipo + número de documento** (CC, CE, pasaporte, TI, PPT). Si vuelve, se reutiliza su registro: no se duplica. El número se normaliza (`1.020.304.050` = `1020304050`).	Historial del cliente, clientes frecuentes y datos limpios para el registro hotelero.	✔ **Sí**
R36	Al reconocer a un cliente se **conserva su nombre registrado**; el teléfono sí se actualiza.	Un error al digitar el documento no le cambia el nombre a otra persona.	✔ **Sí**

Anexo B · Guion de la demo (15 minutos)

Habitación	Huésped	Escenario que deja el seed
101	Carlos Gómez	Alojado, con 3 consumos en su folio → para hacer check-out
201	Ana Martínez	Llega hoy, 3 huéspedes (1 persona adicional) → para check-in
202	Luis Herrera	Llega mañana; la habitación está en limpieza
301	Sofía Ramírez	Llega en 10 días → cancelación gratuita
102	Pedro Castillo	Nunca llegó → el sistema lo marca como no-show
302	Valentina Ortiz	Estadía cerrada (alojamiento + desayunos, pagó con tarjeta)
302	Jorge Peña	Cancelación tardía con penalidad

#	Paso	Tiempo	Qué mostrar
1	El problema	1 min	La habitación tratada como mesa; lo que hoy no existe.
2	El día del recepcionista	2 min	Panel Hoy; marcar lista la 202; check-in de Ana con registro hotelero.
3	Calendario	1 min	Habitaciones × días; el día de salida queda libre.
4	Reserva con tarifa real	3 min	Fechas sobre un puente festivo, 3 huéspedes, desglose, pasaporte exento de IVA, cliente reconocido.
5	No vender dos veces	2 min	Reserva solapada rechazada; explicación de la restricción de PostgreSQL.
6	Folio y check-out	2 min	Cargar y anular consumos de Carlos; liquidación con medio de pago.
7	Cancelaciones y no-shows	1 min	Pedro como no-show; Sofía gratis; Jorge con penalidad.
8	Control (admin)	2 min	Historial de auditoría, usuarios, edición de tarifas y temporadas.
9	Cierre	1 min	Invariantes, hoja de ruta y calidad.

Preguntas probables

- **¿Qué pasa si dos recepcionistas reservan la misma habitación a la vez?** Solo una reserva se crea: lo garantiza PostgreSQL, no el código. Lo mismo aplica a dos check-in simultáneos y a un consumo cargado durante el check-out.
- **¿Cómo se integra con Lazaro (.NET)?** Por la API HTTP documentada en `/docs`; el POS de Lazaro sería un adaptador que carga consumos al folio.
- **¿Y si el hotel cambia las tarifas?** Aplica a las reservas nuevas; las existentes conservan el precio pactado.
- **¿Y si el huésped se va antes?** Se cobran solo las noches usadas, con su temporada, y las restantes quedan libres para vender.